CHIMERA — presenter's guide

Everything you need while showing the demo · BCSE303P Operating Systems Laboratory · Adithyan C S S, Vishal B S, Madhan A

Before you start

Do these three things ten minutes before the review, not during it.

1. Open PowerShell in the project folder and run the demo **once**. The first run is slow because Python loads its libraries; the second is fast.
2. Clear the screen with `cls` so she sees a clean terminal.
3. Have the command already typed, so you only press Enter.

```
# the only command you need
$env:PYTHONPATH="."; python demo_run.py
```

It takes about three seconds and prints six sections. Below is what each one means and what to say.

Why the demo and not the whole project

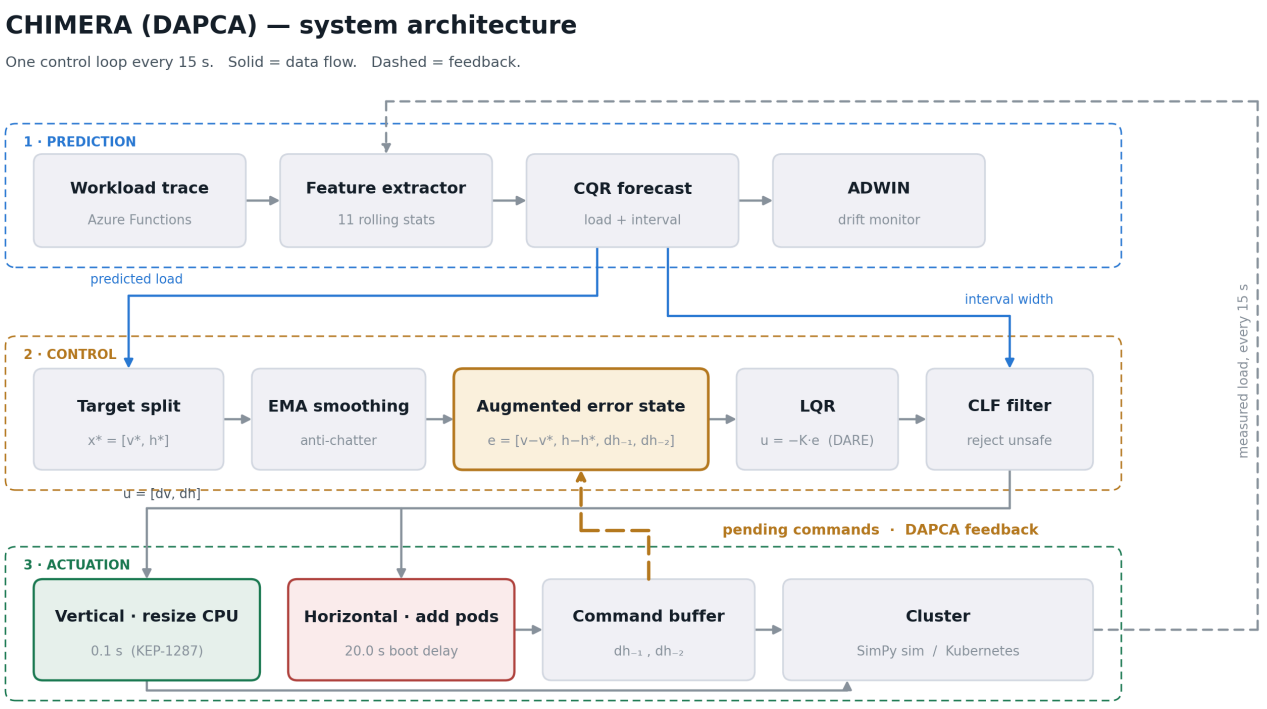
She may ask this. The answer is simple and it is a good answer:

- **It is the same program.** The demo loads the same controller, the same simulator and the same settings as the full evaluation. It is one slice of that evaluation, not a different or simplified version.
- **The full run takes several minutes and shows nothing.** It runs 720 simulations and prints a progress log. There is nothing to look at while it works.
- **It would destroy our results.** The full run overwrites `results\results.csv`. Our verified results are already saved in `results\results_FINAL_verified.csv` and every number in the report comes from there.
- **The two answer different questions.** The demo shows the system *works*, live, in three seconds. The saved results show it works *at scale* — 27 controller configurations, 9 workloads, 3 seeds.

IF SHE ASKS

"The demo runs the same code as our full evaluation — it's one workload out of the 720 runs we did. We're showing this one because it finishes in three seconds and you can actually watch it. The full sweep takes several minutes, only prints a progress log, and would overwrite the saved results our report is built on."

The architecture



Read it in three bands, top to bottom. Everything runs once every 15 seconds.

BAND	WHAT IT DOES
1 · Prediction	Looks at recent traffic and forecasts what is coming next — as a **range**, not a single number, so the system knows how sure it is. ADWIN watches for the traffic pattern changing completely.
2 · Control	Works out how much CPU and how many pods we should have, then computes the cheapest move to get there. The safety filter blocks any move that would make the system less stable.
3 · Actuation	Carries out the move. Resizing CPU takes 0.1 seconds. Adding a pod takes 20 seconds.

The two arrows that matter

The **amber dashed arrow** going back up is our contribution. Pods we have already ordered but that have not booted yet get fed back into the controller, so it does not order the same capacity twice.

The **blue arrows** carry the forecast down into the controller. The second one carries how *unsure* the forecast is — when the forecast is shaky, the safety filter becomes more careful.

SAY THIS

"Three stages running in a loop every fifteen seconds. We predict the load, we decide how much CPU and how many pods, and we act.

The important arrow is this amber one. When we order pods, they take twenty seconds to start. This arrow feeds those pending orders back into the controller, so it knows capacity is already on the way and doesn't order more. That's the part that's new."

The datasets

Nine workloads. Five are real traffic traces from Microsoft's public Azure Functions 2019 dataset; four we wrote ourselves to stress specific weaknesses.

WORKLOAD	TYPE	SHAPE OF THE TRAFFIC	WHY WE INCLUDED IT
w_0001, w_0003	Real (Azure)	Spike	Sudden bursts — tests reaction to traffic we did not predict
w_0002	Real (Azure)	Periodic	Regular daily rhythm — tests whether the forecaster is any good
w_0004	Real (Azure)	Ramp	Steady growth — **this is the one the demo uses**
w_0005	Real (Azure)	Noisy, level	Jittery but not growing — tests that we don't over-react to noise
coupled_oscillation	Written by us	Alternating high/low	Deliberately tries to make the two scaling methods fight
drift	Written by us	Pattern changes	Makes the drift detector fire
extreme_overload	Written by us	Saturation	Pushes past what the cluster can serve
stress_uncertainty	Written by us	Unpredictable	Makes the forecast very uncertain

Each trace is 100 minutes long. Minutes 10–50 train the forecaster, 50–70 calibrate it, and only **minutes 70–100 are scored** — so the model is never tested on data it has seen.

BE HONEST ABOUT THIS

The five real traces are **100-minute excerpts** of 14-day traces. The complete Azure dataset has over 10,000 workloads, and the converter script for it is in our repository, but we have not run it yet.

Say it before she finds it: *"These are 100-minute excerpts. Converting the full 10,000-workload dataset is our next step — the script is written, we just haven't run it."*

The comparison table — how to read it

This is the table in the report that lines CHIMERA up against every existing system. The left half says what each system *can do*; the right half says what it actually *achieved* on our five real traces.

What the columns mean

COLUMN	PLAIN MEANING
Horiz. / Vert.	Which levers the system can pull. Horizontal = add pods. Vertical = resize the CPU of pods already running.
One coordinated law	Whether both decisions come out of a single calculation. If two separate programs decide separately, they can fight each other.
Delay-aware	Whether it knows that pods take 20 seconds to start, and accounts for orders already placed.
Uncertainty model	Whether it knows how confident its own forecast is, instead of trusting a single guessed number.
SLO viol / p99 / pod-min / actions	Measured results. Lower is better on all four.

What each row is

Kubernetes HPA	The industry default. Adds pods only, and only after load has already arrived. Slow to react, so it over-provisions — 215 pod-minutes, the second highest in the table.
Kubernetes VPA	Resizes CPU only, with a fixed number of pods. Fast, so its tail latency is the best in the table — but it can never add pods, so once its fixed pods saturate it stays saturated. That is why its violation rate is still poor at 16%.
HPA + VPA together	Both running side by side, uncoordinated. This is what Kubernetes' own documentation warns against.
AAPA	The paper our project builds on. Predicts load and scales pods only. It gets the best violation rate in the table — but buys it with 272 pod-minutes, 85% more than us.
MagicScaler-inspired	A published heuristic that tracks peak traffic. It is the cheapest in the table and by far the worst, at 74% violations — a useful reminder that cheap alone is not a result.
KOSMOS	Our closest competitor — a 2021 published system that also does both kinds of scaling, but with two separate control laws. We reimplemented it ourselves so the comparison is fair.
CHIMERA	Ours. The only row with a tick in all three capability columns.

The honest reading

We do **not** win every column, and you should say so before she notices. AAPA has a better violation rate. VPA has a better tail. What no other row has is *all four at once*:

- **AAPA** beats us on violations by 1.1 points, and spends 272 pod-minutes against our 147 to do it.
- **VPA** has a better tail, but cannot add pods at all — it would collapse on a larger workload.
- **KOSMOS** is cheaper than us, but its tail is 6× worse and it makes **261 scaling changes** against our 13. In a live cluster that is constant thrashing.
- **MagicScaler** is cheapest and fails three quarters of the time.

CHIMERA is the only system in the table that is never bad at anything. That is the claim — balance, not a clean sweep.

SAY THIS

"The left half of this table is what each system is capable of; the right half is what it actually achieved on our five real traces.

We're the only row that ticks all three capability columns — both scaling directions, one coordinated decision, and awareness of the startup delay.

On results we don't win everything, and I want to be clear about that. AAPA gets a slightly better violation rate, but it spends 85% more resource to get there. VPA has a better tail, but it can't add pods at all, so it would fall over on a bigger workload. KOSMOS is cheaper than us, but its tail is six times worse and it makes 261 scaling changes per run where we make 13.

So our claim isn't that we beat everyone at everything. It's that we're the only one that's never bad at anything."

IF SHE SPOTS THE IDENTICAL ROWS

The **HPA** and **HPA + VPA** rows show exactly the same numbers. That is a real issue we found in our own audit: in this configuration the vertical loop never activates, because the CPU each pod needs stays below the minimum we allow. So the row is currently measuring plain HPA.

Say it plainly: *"We found that ourselves — the vertical loop isn't firing in that configuration, so that row is currently just HPA. It's on our fix list, and it matters because that baseline is meant to demonstrate the loop-fighting problem."* Do not pretend the rows agreeing is a result.

The second table — related work we did not run

Those four rows are not measurements. They exist to show we know the field and to say precisely how we differ from each:

- **GKE Multidimensional Pod Autoscaler** — Google's product. It avoids the loops fighting by giving each one a *different* metric (one watches CPU, the other memory). It never coordinates both on the same resource, which is exactly the problem we solve.

- **ADAPT** — also delay-aware, but pods only, and it solves a fresh optimisation every single step. Ours computes the gain once, up front.
- **OptScaler and BACC** — also use confidence intervals, but again pods only. We feed the interval into a stability constraint; they feed it into a budget.
- **The two patents** — both pick *one* axis or the other using threshold rules. Neither drives both at once from a control law. This is what keeps our contribution clear of prior art.

Reading the terminal, section by section

Section 0 — Setup

```
workload          w_0004  (RAMP, real Azure Functions trace)
scored window     minutes 70-100  (30 min, peak 435 req/min)
SLO               500 ms per request
actuator delays   horizontal 20.0 s  |  vertical 0.1 s   -- a 200x asymmetry
seed              42  (identical for all three controllers)
```

What it is: the rules of the experiment. The promise we are trying to keep is that every request finishes in under half a second. The two delays are the whole problem: adding a pod takes 20 seconds, resizing CPU takes a tenth of a second. The same random seed is used for all three controllers, so nothing that follows is luck.

SAY THIS

"One real Azure trace. We only score the last thirty minutes, because the earlier part trains the forecaster. Our target is every request under 500 milliseconds. And here's the problem in one line — adding a pod takes twenty seconds, resizing CPU takes a tenth of a second. Same random seed for all three controllers, so any difference you see is the control law, not chance."

Section 1 — The control law

```
Augmented state transition matrix A_aug   4x4
    [  1.0   0.0   0.0   0.0 ]
    [  0.0   1.0   0.0   1.0 ]
    [  0.0   0.0   0.0   0.0 ]
    [  0.0   0.0   1.0   0.0 ]

Riccati solution P     eigenvalues [ 3.11  6.63 30.05 53.01]
Positive definite      yes -- the controller is provably stable
```

What it is: the maths running live. A normal controller would use a 2×2 matrix — one row for CPU, one for pods. Ours is 4×4. The two extra rows are a conveyor belt that carries orders we have placed but that have not arrived yet.

The eigenvalues all being positive is not decoration. Our stability proof treats `V = eᵀPe` as the system's "energy" and shows it always decreases. That only works if every eigenvalue of P is positive — so this line is the proof checking itself.

SAY THIS

"This is the control law being solved. A standard controller uses a two-by-two matrix — one row for CPU, one for pods. Ours is four-by-four, and those two extra rows track the orders that are still in flight.

And these eigenvalues are all positive, which is what makes our stability proof valid. It's checking itself every time it runs."

Section 3 — The results

```
controller          SLO viol   mean RT    p99 RT   pod-min  actions
CHIMERA               12.90%     0.506     8.711     123.2       23
CHIMERA (no SA)       14.49%     0.557     9.238     115.0       23
HPA                   17.69%     0.631     9.961     270.8       20

Ablation: switching delay-state augmentation off costs +1.59 points of SLO
```

What the columns mean, in plain words:

COLUMN	MEANING
SLO viol	How often we broke our promise. 12.90% means roughly 1 request in 8 took longer than half a second. Lower is better.
mean RT	Average time a request took, in seconds.
p99 RT	The unluckiest 1% of users. 8.7 seconds is their wait.
pod-min	Pods multiplied by minutes. A rough measure of how much we used.
actions	How many times we changed something. Every change disturbs a live system, so lower is calmer.

The two comparisons that matter. Against HPA we are 4.8 points better. Against *ourselves with the delay-tracking switched off* we are 1.59 points better — and that second one is the proof that our contribution does something, because everything else in that run is identical.

ONE THING YOU MUST NOT SAY

Do **not** say we use half the resources. The `pod-min` column counts pods, not CPU — and our pods are bigger. Measured properly:

```
CHIMERA   151.5 CPU-minutes  ·  4.1 pods avg  ·  1.24 cores each
HPA       135.4 CPU-minutes  ·  9.0 pods avg  ·  0.50 cores each
```

In actual CPU we use **12% more**, not less. The correct claim is: **about the same CPU, less than half the pods, and 4.8 points better.** Fewer pods still matters — every pod avoided is a 20-second cold start avoided.

Section 4 — What the cluster did



What it is: a tiny graph of the whole run. Each line is one measurement over time, taller meaning more.

The point is the second HPA line — it is completely flat, because HPA *cannot* change CPU. It only has one lever, so it pulls it hard and ends up with ten pods. We have two levers, so five pods is enough.

SAY THIS

"These are mini graphs of the whole run. Look at HPA's bottom line — it's flat, because HPA can't change CPU at all. It only knows how to add pods, so it climbs to ten.

Ours moves both. That's why we only need five pods. And every pod we don't create is a twenty-second startup we don't have to wait for."

Section 5 — The contribution, step by step

```
  t (s)  replicas   cpu/pod   pending commands
    30         3      0.60   [+1, +0]
    45         3      0.90   [+0, +1]
    60         2      1.00   [-1, +0]
    75         4      1.00   [+2, -1]
    90         4      1.00   [+0, +2]
```

What it is: each line is one decision the controller made while orders were still in flight. The two numbers in brackets are the conveyor belt — what was ordered one step ago, and two steps ago.

Take the line at 75 seconds. The cluster has 4 pods right now. But the buffer says `+2` — two more were ordered fifteen seconds ago and are still booting. So the controller effectively has 6 pods coming, and it does not order any more.

A controller without this sees only "4 pods, not enough" and orders more. Twenty seconds later all of them arrive at once, it is now over-provisioned, and it starts removing pods. That back-and-forth is what our 1.59 point improvement removes.

SAY THIS

"This is the actual mechanism. Each line is one decision, and the numbers in brackets are what we've ordered but not yet received.

Look at seventy-five seconds. Four pods running, but two more already on the way. So we wait. A controller without this would see four pods, think it's short, order more — and twenty seconds later everything arrives at once and it has to undo it. That's the oscillation we remove, and it's worth 1.6 points on this trace."

Three questions she will probably ask

"If it works, why are 12.9% still failing?"

Because we aim for 70% CPU usage, and at that level the average request takes 0.506 seconds — which is right at our 500ms limit. Running at lower usage would cut failures but cost more machines. The fair test is against HPA aiming at the same 70%, and there we are 4.8 points better.

"Why is one step 5 milliseconds when your report says under 0.5?"

Be straight: the code currently re-solves the control equation every step instead of solving it once and saving it, and it runs the safety search twice for measurement. Both are on our fix list. Then add the perspective — 5 milliseconds inside a 15-second loop is 0.03% of the interval, so it makes no practical difference either way.

"Has this run on a real Kubernetes cluster?"

No, and say so. The Kubernetes pieces are written — the custom resource, the permissions, the container — but the part that reads live traffic still returns test numbers. Everything measured so far is simulation. Deploying on a

real cluster is our next milestone.

All figures measured from the project's own code on 23 September 2026. The demo uses the same controller, simulator and settings as the full 720-run evaluation saved in `results\results_FINAL_verified.csv`.